Objects

CCS1200
Introduction to Object-Orientation

Dr Ioanna Stamatopoulou

What is an Object?

- An **Object** is the concrete realisation of a class description
  - Also called an **instance of a class**
- **All objects are instances of a class**

Objects: Identity, State & Behaviour

- An object in OOP is described in terms of:
  - its **identity**: What is it called?
  - its **state**: What does it "look" like?
  - its **behaviour**: What does it do?

  ... as objects in the real world

Object State

- The **state** of any object is described by the **values** of its **attributes**
- A rectangle's state:
  - identity: rectangle1
  - state: width: 10.2, height: 5.4
  - behaviour: getHeight, setWidth, getArea, getPerimeter, etc.

- Different rectangles can have different values for their attributes (**different states**):

rectangle1	**rectangle2**
width= 10.2	**width**= 8.4
height = 5.4	**height** = 15.7

Object Behaviour

- How do we make an object do something useful?
  - We make a request by sending a message
- The behaviour of an object, i.e. the method used by an object in response to a message, is determined by its class
  - All objects of a class use the same method (behave in a similar fashion) in response to similar messages

Object Behaviour (cont'd)

- What requests can I make to an object?
  - These are **defined by its public interface**

```
Rectangle r1= new Rectangle();
r1.setHeight(10.1);
```

Class Name

Rectangle
-width: double -height: double
+getHeight(): double +getWidth(): double +getArea(): double +getPerimeter(): double +setHeight(height: int): void +setWidth(width: int): void +isSquare(): boolean

Public Interface:
The set of all public (+) elements
(attributes or methods)

Object Declaration

- Only after a class has been defined, we can declare, create and use its objects
- Object declaration is similar to variable declaration
  - int x;

```
//declaration of an object
Rectangle  r1;
```

class name (type)

object name

Object Creation

- For the creation of an object we use the new keyword and call the constructor of the object's class

```
//creation of an object
r1 = new Rectangle();
```

object name

becomes a new

Rectangle():
This is a call to the class constructor: a method with the name of the class that creates objects

  - We will discuss constructors promptly!

Message Passing

- Message passing is implemented as method calling

```
// message passing
r1.setHeight(10.1);
```

object name dot method call arguments

Controlling Access

Access modifiers

- There are two access modifiers:
  - **public** variables or methods are accessible wherever the program has a reference to an object of the class
  - **private** variables or methods are accessible only inside the class where they are declared
- **static** attributes and methods are accessible through the class
  - This means that static attributes' values are common to all created objects
  - There is only one copy maintained for all objects of a class

Use of Access Modifiers

- Typically:
  - attributes are declared private
  - methods are declared public
- It is also possible to have public attributes but uncommon and it is a bad and dangerous programming practice
- It is possible to have private methods, which are utility or helper methods, because they can be called only by other methods of this class and are made to support the operation of those methods (see next slide)

Time class: Private methods

```
public String toStandardString() {
    int hourDisplayed = calculateHour();
    String pm_am = find_pm_or_am();

    return hourDisplayed + ":" + minutes + ":" + seconds
                                          + " " + pm_am;
}
```

```
private int calculateHour(){
    if (hour == 0 || hour == 12) return 12;
    else return hour%12;
}

private String find_pm_or_am(){
    if (hour <12) return "AM";
    else return "PM"
}
```

Private methods (not part of the public interface) used only internally by the toStandardString() method

Attribute Access

- **Inside** a class A, the attributes of class A are **accessible everywhere**
  - They are declared **inside the class** and outside the methods so they are **accessible to all of the class's methods**, no matter what their access modifier is

Attribute Access (cont'd)

Outside a class A, the attributes of class A are **only accessible**:

- If they are declared as **public**, through the object's name and the dot notation: **objectName.variableName**
  - e.g. the Array is a class that has a public attribute named length

```
int a[] = new int[4];
int b = a.length;
```

- If they are declared as **private**, only **through the public set and get methods**
  - e.g. if we have a Time object timeObj and we want to access its minutes attribute:

```
int min = timeObj.getMinutes();
```